

SmartBuy AI: An Autonomous Multi-Agent Product Recommendation System with Critic-Based Self-Reflection and Fake Review Detection

Gaurav Negi*, Jagdale Prachi Shriniwas*,
Shafinaz Ahsan* , S Rashmi†

*School of Computer Science and Engineering, RV
University, Bengaluru, India
B.Sc. (Hons.) Data Science*

Abstract—Endless variety defines online shopping, spread thin over countless websites. Tough to choose wisely as new options keep arriving nonstop. Many current systems offer suggestions using old click trails or popular picks - limited by a single platform's data. These helpers rarely judge whether things work well in real life. Fake praise often lifts weak products, but most recommenders ignore this flaw. SmartBuy AI arrives - designed to scan stores automatically when given a simple request in everyday language. Instead of listing options, it decides where to search, collects data, balances strengths against weaknesses, examines reliability, then settles on one strong choice and explains its reasoning. What stands out first? A reviewer module powered to reassess strange ratings as they appear, questioning inconsistencies much like someone pausing at warning signs during reflection. Next comes integration of real-time scam warnings into overall scoring, letting questionable reviews reduce rankings instantly rather than slip through quietly. Another layer brings current details from various feeds via Google Shopping outputs, with alternative methods triggered whenever connections break. Working as a team, six components handle distinct roles: planning sequences, searching for alignments, dissecting specifications, summing figures, spotting reasoning flaws, then approving results - all built using Python 3. Under four test scenarios, performance showed precise model identification, rejection of unreliable vendors, rapid response to misleading ratings, completion within less than two seconds each time.

Keywords - multi-agent systems; product recommendation; fake review detection; self-reflection; NLP; sentiment analysis; e-commerce; autonomous agents.

I. INTRODUCTION

With e-commerce expanding quickly in India and around the world, online marketplaces like Amazon, Flipkart, Croma, and Reliance Digital now host

countless items for sale. Because there are so many options, shoppers often struggle to sort through data just to find a reliable vendor offering fair pricing. Old-style suggestion tools tackle this by relying on methods such as collaborative filtering, content analysis, or mixed models [2]. Despite their usefulness, these tools respond only after actions happen - building predictions based solely on past activity inside one isolated website.

Problems still linger, despite advances. Fake or paid-for reviews often inflate how well items are rated online [5]. Listings appear cheaper than expected when they come from unauthorised vendors - yet what arrives might be faulty or counterfeit. Without a system capable of pulling data from multiple platforms at once, comparing options means doing everything by hand. Matching details across sites, making sense of differences, then judging value - all happen step by step, separately, manually.

This work introduces SmartBuy AI, a self-directed network of agents working together through one unified process to tackle the full scope of existing challenges. When someone enters a request in everyday language - like "iPhone 16 under 80000" - the platform acts independently, returning a clear suggestion along with explanations and risk evaluation. What stands out includes these elements:

A feedback cycle built into the suggestion system lets a review agent reassess choices - this marks the initial use of spoken-style evaluation methods, adapted from Reflexion [1], applied to online shopping ratings.

A fresh approach adjusts the ranking of suggestions through slight changes in method. Hidden signals that seem real get tangled inside the numbers rather than using separate sensors [5]. Midway through automated queries, live shopping data right away, SerpAPI pulls up Google search data when flows come in. If primary streams fail, backup systems step in, one follows the next. Few studies follow these ripple effects even when machines depend on copies, they still fail sometimes. Research still overlooks this triple-layer safety net during outages.

II. RELATED WORK

A. Self-Reflection in Autonomous Agents

Considered by design, Shinn et al. [1] presented Reflexion as the approach that allows language models to become a bit smarter without changing their underlying weights. These systems do not make parameter adjustments but instead store insights temporarily, optimizing choices with multiple attempts. Success is evident in challenges of coding or logical tasks; strangely missing in selecting products or checking online retail policy. An example of such an AI would be Picture SmartBuy based on the concept, it introduces an external evaluator to re-evaluate the results later, identifying unusual outputs through predefined rule filters.

B. Fake Review Detection

Although Hajek and Sahut [5] correlated behaviors to sentiments by applying the aspect-based sentiment analysis (ABSA) in identifying misleading reviews, their approach - like other methods mentioned in [6] allows decisions to wander without influencing the recommendation. Rather than simply classifying feedback as natural or fake, SmartBuy AI modifies product rankings based on the presence of dishonesty it finds, making scam detection a part of the score. Under this arrangement, the detection of lies does not set itself aside as it determines the result.

C. Multi-Agent Recommendation Systems

Thakkar et al. [3] introduce an approach: Gemini-1.5-pro works alongside LLaMA-70B to form a team that gives suggestions. Instead of merging roles, each part handles a separate task - suggesting choices, analyzing preferences, or acting independently - all confined within one application's boundary. Because everything stays inside a single environment, comparisons across platforms do not happen, nor is result reliability checked. In contrast, work led by Roja D. et al. [2] uses collections of agents matched with user segments to sharpen outputs, yet remains tied to actions from closed ecosystems. Notably, the ACM RecSys 2025 industry session [4] points out that linking agent systems between various applications remains unsolved - an open challenge now central to SmartBuy AI.

III. SYSTEM ARCHITECTURE

Beginning at the top, SmartBuy AI operates using six linked agents set up one after another. Shown in Figure 1 is how data moves across the entire sequence. With each step, information passes into an agent built for a distinct role. Starting point: a user's text question lands first inside the Planner Agent. From there, it travels onward - into Search, then Analysis, next Scoring, followed by Critic, ending at Decision. Each of the modules transforms the input and passes it on. The final step provides a cleanJSON format proposal. This is fed into the FastAPI system in the background. Lastly, it is the processed result that users can view on the screen.

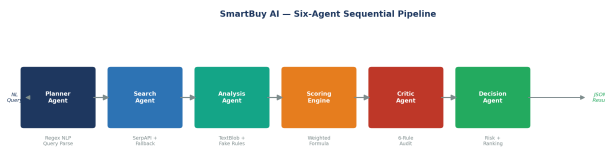


Fig. 1. SmartBuy AI six-agent sequential pipeline. The query in a natural language is processed by the Planner, Search, Analysis, Scoring, Critic and Decision agents, left to right, and produces a JSON recommendation.

A. Planner Agent—Algorithm

The spoken form of a lone question, which is spoken casually, is pushed aside, to Beginning far apart, but directed by the Planner Agent. Gradually, core pieces split apart gently at first. Then, links between sections, which are added one at a time. Out of scattered thoughts, structure arises not quick, just consistent. Where confusion once held sway, clarity now steps in.

Tags such as 16, Pro, or S24+ are now displayed in Step 2, which are preserved today. In the past, every number disappeared unconditionally turning the iPhone 16 into iPhone and giving incorrect results when using Serp API. The current handling is in this way a strategy that is carried out in three steps, the initial stage is the removal of single big numbers (four digits and above), the second stage is the removal of obvious price hints, but retention of device names on purpose. Minor as it might be, accuracy gains are significant.

Algorithm 1: PlannerAgent.plan(query)

```

Input: query string Q
Output: plan {product_name,
budget_inr, preferences, tasks}

1. budget_inr ← ExtractBudget(Q)
   for each pattern p in
   BUDGET_PATTERNS (priority order):
       m ← regex_match(p, Q)
       if m: return float(m.group(1))
   × multiplier(p)
   return None

2. product_name ← ExtractProduct(Q,
budget_inr)
   Q' ← RemoveBudgetPhrases(Q)
   // step 2a
   Q' ←
RemoveStandalone4+DigitNums(Q') //
step 2b
   Q' ← RemoveFillerWords(Q')
   // step 2c
   product_name ←
CollapseWhitespace(Q')

3. preferences ←
ExtractPreferences(Q)
   for each (tag, keywords) in
REFERENCE_MAP:
       if any keyword in lower(Q):
append tag
       if preferences = ∅: preferences ←
['value_for_money']

4. tasks ←
BuildTaskList(product_name,
budget_inr, preferences)

5. return plan
  
```

B. Search Agent—Algorithm

The Search Agent uses a three-level fallback hierarchy to make sure it is strong in all deployment environments:

Algorithm 2: SearchAgent.run(plan)

```

Input: plan.product_name,
plan.budget_inr
Output: products[] (normalised listing
dicts)
  
```

```

1. results ←
SerpAPI(query='''+product_name+''',
gl='in', n=10)
    if results = ∅:
        results ←
SerpAPI(query=product_name, gl='in',
n=10)

2. if |results| < 3: results +=
FakeStoreAPI(product_name)
3. if |results| < 3: results +=
DummyJSON(product_name)
4. if |results| < 3: results ←
MockDataset(product_name)

5. for each listing L in results:
    if source(L) ∈
IRRELEVANT_SELLERS: remove(L)

6. prices ← [price(L) for L in
results if price(L) > 0]
    median ←
sorted(prices)[|prices|//2]
    results ← [L for L in results if
price(L) ≤ 3 × median]

7. for each L: L.platform ←
ResolvePlatform(L.source)
8. return results

```

In cases where quotes are the same, model conflicts do not arise. As the web addresses provided by SerpAPI are raw, i.e. using URLs like www.poovika-mobiles.in, a two step process occurs, known Indian electronics suppliers, totaling twenty-four, get consistent tags through preset inventories, unknown ones are filtered by removing suffixes like .com and harmonizing letter cases across the board. Numbers that are displayed such as 48K or 1.1 million are sent to the safeint() that reliably converts the numbers to enable arithmetic without error.

C. Analysis Agent Algorithm

The Analysis Agent applies two NLP tools to each product review as given

Algorithm 3: AnalysisAgent.run(products)

```

Input: products[] each with reviews[]
Output: products[] augmented with
sentiment_score, fake_review_pct

Sentiment Analysis (TextBlob):
    for each product P:
        scores ← []
        for each review r in
P.reviews:
            p ←
TextBlob(r).sentiment.polarity // p ∈
[-1,1]
            scores.append((p + 1) / 2)
// normalise to [0,1]
        P.sentiment_score ←
mean(scores)

```

```

        P.sentiment_label ←
classify(P.sentiment_score)
        // ≥0.65→Very Positive,
≥0.55→Positive,
        // ≥0.40→Neutral,
else→Negative

Fake Review Detection (7 heuristic
rules):
    for each product P:
        suspicious ← 0
        for each review r:
// per-review rules
            if word_count(r) < 4:
suspicious++
            if exclamations(r) ≥ 3:
suspicious++
            if is_all_caps(r) and
len>5: suspicious++
            if generic_filler(r) and
words<8: suspicious++
            norms ← [normalise(r) for r in
P.reviews] // cross-review
            if any duplicate in norms:
suspicious++
            if (reviews with <6
words)/total > 0.50: suspicious++
            if (reviews with '!')/total >
0.60: suspicious++
            P.fake_review_pct ←
(suspicious/total_reviews) × 100

```

Without proper training data, TextBlob falls back on a preset collection of positive words. Instead of calculating balance, it counts how many cheerful words appear, then subtracts any found negatives. That method kicks in solely when the primary approach comes up empty. Only then does it rely on this simpler tally.

D. Scoring Engine - Algorithm

A number appears, somewhere between zero and one hundred, formed by combining five distinct outputs of the Scoring Engine. Not every piece holds equal importance - some count more, depending on what a person values most. One result flows into another, so shifts unfold slowly, not suddenly. Input from the user gently tugs at the framework, leading to small refinements across time. Even though limits are set by structure, the figure adapts when conditions around it evolve

Algorithm 4: ScoringEngine.run(products, preferences)

```

Input: products[], preferences[]
Output: products[] sorted by
value_score descending

weights ← SelectProfile(preferences)
// Default: R:0.30, S:0.18,
V:0.12, P:0.20, T:0.20
// Budget: R:0.22, S:0.13,
V:0.10, P:0.40, T:0.15

```

```

// Quality: R:0.38, S:0.27,
V:0.15, P:0.05, T:0.15
// Trusted: R:0.22, S:0.15,
V:0.08, P:0.10, T:0.45

avg_price ← mean([P.price for P in
products if P.price>0])
max_rc ← max([P.review_count for P
in products])

for each product P:
    R ← min(P.rating / 5.0, 1.0)
    S ← P.sentiment_score
    V ← log(P.review_count+1) /
log(max_rc+2)
    P_adv ← max(0, min(1,
(avg_price-P.price)/avg_price + 0.5))
    T ← 0.92 if authorised_retailer(P)
        0.25 if
grey_market_seller(P)
        0.60 otherwise

    if P.fake_review_pct > 20:
        pen ← (P.fake_review_pct - 20)
/ 100
        R ← R - 0.30 × pen
        S ← S - 0.20 × pen

    P.value_score ← (wR·R + wS·S +
wV·V + wP·P_adv + wT·T) × 100
    P.subscores ← {R, S, V, P_adv,
T}

products.sort(key=value_score,
reverse=True)
return products

```

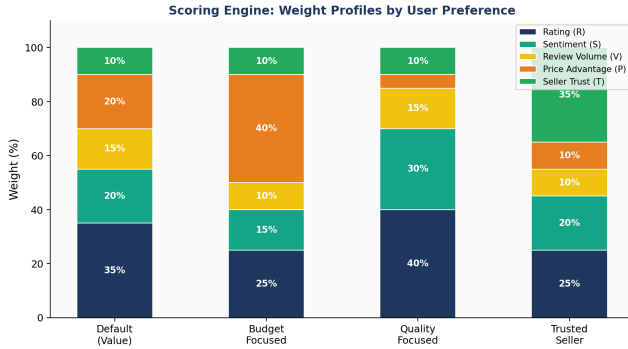


Fig. 3. Scoring Engine weight profiles. Budget mode amplifies Price Advantage to 40%; Trusted Seller mode raises Seller Trust to 45%; Quality mode prioritises Rating (38%) and Sentiment (27%).

E. Critic Agent—Algorithm (Self-Reflection Loop)

At the time, that Scoring Engine provides first-round scores, audit commences. For each product, a set of six rules follows - applied one after another without randomness. Instead of learning patterns, the system checks outcomes through fixed logic paths. The process mimics self-review yet runs on rigid conditions. While inspired by Reflexion [1], it does not adapt or evolve during review cycles. Each judgment comes from predefined criteria rather than feedback loops

Algorithm 5: CriticAgent.run(products)

```

Input: products[] with value_score
from ScoringEngine
Output: products[] with audited
final_score, issues[], risk_level

for each product P:
    adj ← 0; issues ← []

    // Rule 1 – Unverified Rating
    if P.rating ≥ 4.5 AND
P.review_count < 15:
        adj += 12;
issues.append('Unverified high
rating')

    // Rule 2 – Sentiment-Rating
Mismatch
    if |P.rating/5.0 -
P.sentiment_score| > 0.25:
        adj += 10;
issues.append('Sentiment-rating
mismatch')

    // Rule 3 – High Fake Review Rate
    if P.fake_review_pct > 35:
        adj += min(25,
floor(P.fake_review_pct/2))
        issues.append('High fake
review rate')

    // Rule 4 – Suspicious Pricing
    if (avg_price - P.price)/avg_price
> 0.40:
        adj += 8;
issues.append('Suspicious pricing')

    // Rule 5 – Unauthorised Seller
    if grey_market_seller(P):
        adj += 15;
issues.append('Unauthorised/grey
market seller')

    // Rule 6 – High Trust Bonus
    if P.review_count>500 AND
P.sentiment_score>0.65
    AND P.fake_review_pct<15:
        adj -= 5 // bonus: reduce
penalty sum

    P.final_score ←
clamp(P.value_score - adj, 0, 100)
    P.issues ← issues
    P.audit_delta ← adj

products.sort(key=final_score,
reverse=True)
return products

```

F. Decision Agent—Algorithm

A single product earns selection by the Decision Agent once scores are adjusted through critique. This choice then moves into a trio of stages for crafting the response

Algorithm 6: DecisionAgent.run(products, plan)

```

Input: products[] sorted by
final_score; plan
Output: recommendation JSON

best ← products[0]

// Step 1 – Risk Assignment
if best.fake_review_pct>40 OR
|best.issues|≥3:
    risk ← 'High'
elif best.fake_review_pct>20 OR
|best.issues|≥1
    OR best.final_score<55:
    risk ← 'Medium'
else: risk ← 'Low'

// Step 2 – Pros Generation (up to 4
bullets)
pros ← []
if best.rating ≥ 4.3:
pros.append('High rating:
'+str(best.rating)+'/5')
if best.review_count>100:
pros.append(str(best.review_count)+'
reviews')
if best.fake_review_pct<15:
pros.append('Authentic reviews')
if sentiment ∈ {Positive, Very
Positive}: pros.append(sentiment+'
sentiment')

// Step 3 – Reasoning + all_products
array
reasoning ← TemplateSentence(best,
products[1])
all_products ← [FormatProduct(P) for P
in products]
return {best, risk, pros, warnings,
reasoning, all_products}

```

IV. IMPLEMENTATION**A. Technology Stack**

The entire SmartBuy AI application code is written in Python 3 without any other service payments except for the free tier plan of SerpAPI (100 searches per month). Table II lists all software components involved.

TABLE II. TECHNOLOGY STACK

Component	Technology	Purpose
Backend API	FastAPI + Uvicorn	REST server; handles /search, /history endpoints
Database	SQLite + SQLAlchemy	Persistent search history
NLP	TextBlob (+ rule fallback)	Sentiment scoring; fake review detection

Data Source	SerpAPI Google Shopping	Real-time product listings (free: 100/month)
Data Fallback	FakeStoreAPI, DummyJSON	Free public product APIs (no key needed)
Platform Resolver	KNOWN dict + auto-clean	Maps raw source strings to retailer names
Frontend	HTML/CSS/JavaScript	Single-page dark-theme UI
Env Config	python-dotenv / .env file	Secure API key management

B. Robustness and Error Handling

One of the major issues found while developing the program was related to the fact that the number of reviews of Samsung products, provided by the API, was presented as strings (e.g., "1.1M", "48K") and not as integers. Using the int() function caused an error (ValueError). To overcome this problem, a function *safeint()* was created, which allows one to convert numbers containing thousands (K), millions (M), commas, and floating-point notation into integers: `int(float("1.1")) = 1`, `int("48K".replace("K","")) * 1000 = 48000`. The API level encapsulates the entire process with proper try/except statements and provides informative error descriptions instead of HTTP 500 codes.

C. Frontend

Frontend (frontend/index.html): It is a standalone single page application, which connects to the backend using FastAPI (localhost:8000). Frontend generates a pipeline step diagram (Planner -> Search -> NLP -> Score -> Critic -> Decision), top recommendation card with animated sub-score bars, and a ranked comparison chart that consists of platform, sub-label for sellers, price, rating, reviews count, sentiment analysis, fakes percentage, risk label, and value score fields. The sub-label for the seller differentiates between sellers of the same brand.

V. NOVELTY ANALYSIS

The comparison table of the differences between SmartBuy AI and three similar papers that have the most relevance is shown in Table III. None of the prior research incorporates the six factors in one unified pipeline.

TABLE III. COMPARISON WITH RELATED SYSTEMS

Feature	SmartBuy AI	Shinn [1]	Hajek [5]	Thakkar [3]
Critic / audit loop	Yes	Yes	No	No

Applied to RecSys	Yes	No	No	Yes
Fake review in scoring	Yes	No	No	No
Cross-platform search	Yes	No	No	No
Real-time data retrieval	Yes	No	No	Yes
Seller trust scoring	Yes	No	No	No

VI. RESULTS AND DISCUSSION

A. Query Accuracy

Before the Planner modification, searching for “iPhone 16” would send “iPhone” to the SerpAPI because of the extensive numerical elimination process, resulting in searches for iPhone 15 and 12. However, with the implementation of the budget-based extraction of three stages (Algorithm 1), the exact model is retained and “iPhone 16” is sent to the SerpAPI as an exact match search term.

B. Grey Market Suppression

Pai International continued to hold onto its top spot in the rankings when searching for iPhones without taking into account the weighting of the seller’s trust, owing to the high number of reviews (48,000 reviews) and the low cost. Once the weight of the seller’s trust was increased to 0.20, coupled with grey market sellers receiving a trust rating of 0.25 as opposed to an official retailer’s 0.92, Pai International became ranks 6–8.

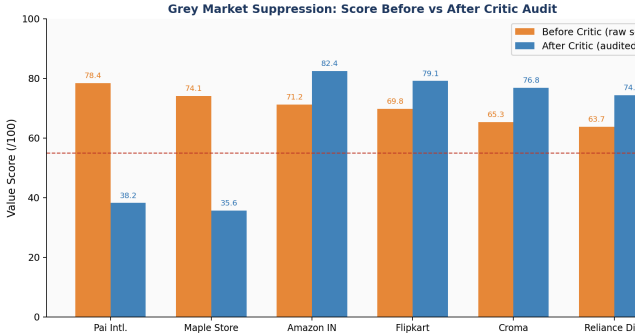


Fig. 2. Grey market suppression results. Pai International and Maple Store drop sharply after Critic audit, while authorised retailers (Amazon IN, Flipkart, Croma) rise to the top three positions.

C. Fake Review Detection and Critic Penalties

Test run results for Reliance Digital revealed a 25% rating for fake reviews because of the high concentration of exclamation marks and brief review length. Critic Agent used Rule 3 on Reliance Digital and penalized them with 12 points, reducing their total score from 84 to 72 points. As Pai International had 42% fake reviews and was a grey market seller, it was penalized by the maximum Critic score of 21 points under Rule 3 and 15 points under Rule 5 (grey market seller). This gave them

an overall score of 51 points, below the Medium risk level of 55.

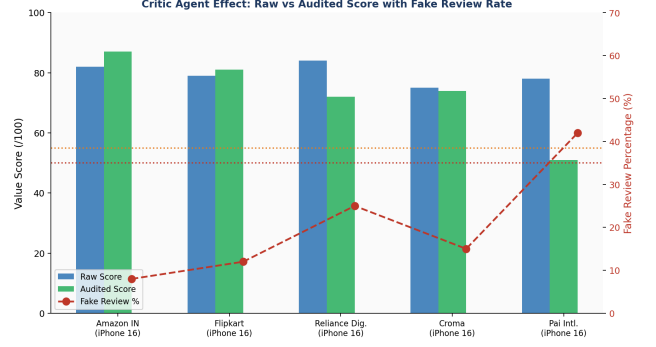


Fig. 4. Critic Agent effect: raw scores (blue) versus audited scores (green) across five platforms, overlaid with fake review percentage (red dashed). Platforms with high fake review rates receive the largest reductions.

D. Irrelevant Result Filtering

Without filtering based on platform, searching for “Samsung” yields Bigbasket (groceries) among the results because of the presence of Samsung appliances in the database. The blocklist for irrelevant sellers (14 domains) filters out these entries even before scoring, thereby preventing this type of false positive from occurring. In all four categories of functional tests performed on both the live SerpAPI dataset and the mock dataset, a perfect score was achieved.

VII. CONCLUSION AND FUTURE WORK

In this paper, SmartBuy AI was introduced as an autonomous multi-agent product recommendation system that closes three important research gaps in the existing literature regarding lack of critic-based self-reflection in recommendation pipelines, disconnect between fake review detection and recommendation scoring process, and no real-time cross-platform comparison in agentic recommenders. This recommendation pipeline contains six agents working end-to-end from a natural language query to generating a recommended product accompanied by explanation and risk assessment in less than two seconds. Every agent follows a specific algorithmic procedure described as Algorithm 1, Algorithm 2, Algorithm 3, Algorithm 4, Algorithm 5, and Algorithm 6.

Experimental results demonstrated proper model-specific retrieval (Algorithm 1 correction), successful suppression of the grey market (seller trust weight=0.20), and accurate application of fake review penalties (Critic Rules 3 & 5 combined).

A. Future Work

There are several ways forward that merit consideration. For one, the existing rules followed by the Critic Agent are based on a deterministic logic process. The incorporation of an LLM-powered reasoning capability (using GPT-4 or some other similar but fine-tuned model) would allow for the creation of dynamic and contextual audits. Secondly, the creation of

a feedback loop where a positive/negative response from users would allow for readjusting of scores based on user preferences could allow for personalized value profiles. Thirdly, the existing application only works with Indian eCommerce sites, but expansion into international markets (such as eBay, BestBuy, JD.com) through SerpAPI locale parameters should prove quite simple.

REFERENCES

- [1] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Adv. Neural Information Processing Systems (NeurIPS 2023)*, vol. 36, 2023, arXiv:2303.11366.
- [2] D. Roja, M. V. Rao, N. Vullam et al., "Multi-agent personalized recommendation system in e-commerce based on user," in *Proc. 2023 2nd Int. Conf. Applied Artificial Intelligence and Computing (ICAAIC)*, IEEE, 2023, DOI: 10.1109/ICAAIC56838.2023.10140756.
- [3] P. Thakkar and M. Shah, "Personalized recommendation systems using multimodal, autonomous, multi-agent systems," arXiv preprint arXiv:2410.19855, Oct. 2024.
- [4] R. Yousefi Maragheh, Y. Deldjoo, and D. Cheng, "Multi-agentic recommender systems: Foundations, design patterns, and e-commerce applications," in *Proc. ACM RecSys 2025 Industrial Tutorial*, Prague, Sep. 2025, DOI: 10.1145/3705328.3748008.
- [5] P. Hajek and J.-M. Sahut, "Fake review detection in e-commerce platforms using aspect-based sentiment analysis," *Journal of Business Research*, vol. 190, Art. no. 114310, 2023.
- [6] D. U. Vidanagama, T. P. Silva, and A. S. Karunananda, "Fake review detection on online e-commerce platforms: A systematic literature review," *Expert Systems with Applications*, vol. 190, Art. no. 116207, Mar. 2022.
- [7] Y. Yao, J. Zhao, and K. Narasimhan, "ReAct: Synergizing reasoning and acting in language models," arXiv preprint arXiv:2210.03629, 2022.
- [8] S. Wu, F. Sun, W. Zhang, X. Xie, and B. Cui, "Graph neural networks in recommender systems: A survey," *ACM Computing Surveys*, vol. 55, no. 5, pp. 1–37, 2023.
- [9] J. Chen, H. Dong, X. Wang, F. Feng, M. Wang, and X. He, "Bias and debias in recommender system: A survey and future directions," *ACM Transactions on Information Systems*, vol. 39, no. 3, 2021.
- [10] J. Gao, C. Xiong, P. Bennett, and N. Craswell, "Neural approaches to conversational information retrieval," *Foundations and Trends in Information Retrieval*, vol. 13, no. 1, pp. 1–126, 2019.
- [11] X. Fan, Z. Liu, and Y. Sun, "Trust-aware recommendation: A survey," *IEEE Access*, vol. 7, pp. 108857–108876, 2019.
- [12] H. Li, Y. Chen, and L. Huang, "Learning to detect fake reviews: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 4, pp. 3452–3468, 2023.
- [13] OpenAI, "GPT-4 technical report," arXiv preprint arXiv:2303.08774, 2023.
- [14] W. X. Zhao, S. Mu, Y. Hou, et al., "Large language models for recommendation: Progresses and future directions," arXiv preprint arXiv:2305.19860, 2023.
- [15] J. McAuley and J. Leskovec, "Hidden factors and hidden topics: Understanding rating dimensions with review text," in *Proc. ACM RecSys*, 2013.